Aiko

Due-Diligence Dossier

An agent that lives on your laptop and does the things you'd do, so you don't have to.

CONFIDENTIAL

Prepared for investor due diligence. Not for distribution. This dossier describes proprietary architecture; please do not share it.

Aiko -

Confidential. *Prepared for investor due diligence. Not for distribution.*
This dossier describes proprietary architecture; please do not share it.

What Aiko is, in one paragraph

Aiko is an agent that lives on your laptop and does the things you'd do, so you don't have to. **Coworkers bombing you on Telegram - Aiko tells you what actually** applies to you and replies in your voice. Your mum's birthday - Aiko finds a restaurant nearby and books a table for six at 6:30. *If you can do it on your laptop, ask Aiko.* Under the hood, Aiko is not a wrapper around a frontier LLM. It is a **fractal energy-based harness**: a substrate that routes, verifies, and learns from doing - grounding a small local model with a live index and a shared, auditable memory, so it gets *twice the reliability at half the cost* of running a large model for computer use.

The three pillars (the pitch)

Pillar	The claim	Honest status (see SStatus legend)
Reliable	Our own Energy-Based model for computer use, trained on-device; twice the reliability, half the cost	Harness + energy router **BUILT+PROVEN**; the "2x / 1/2" figures are **DESIGN targets, not yet independently benchmarked**
Private	Weights on-chain; zero-knowledge credential binding; your data stays yours; Solana volume makes the product pay for itself	Local inference + sealed data + reward economics **BUILT/DESIGNED**; **ZK credential binding is RESEARCH-DIRECTION, not built**
Powerful	A new architecture lets Aiko learn *as it works*	Train-on-fire + energy-ranked resolution **BUILT+PROVEN** as a router; "reasoning-grade intelligence" is a **falsifiable bet, not a result**

We state the limits as loudly as the vision - that is the substrate's own tradition, and this dossier keeps it. Where something is a target or a direction, it says so, in the same breath as the claim.

The bet (why this is a different scaling law)

Frontier AI scales intelligence as **parameters x data x compute** - static, centralized, and *your data trains their model*. Aiko scales it as **index** coverage x collective firing history x routing/verification quality **- live,** distributed, and *your data stays yours*. Knowledge lives in a retrievable index, not in frozen weights, so **the marginal cost of a thought falls toward the cost** of a lookup and a fold**. That is the whole thesis; everything else is mechanism.**

How to read this dossier

#	Document	What it answers
00	**README-DUE-DILIGENCE** (this)	What is it, what's the pitch, how to navigate
01	**PRODUCT-aiko**	What does the user get; the use cases; why it beats a raw LLM for computer use
02	**ARCHITECTURE-fractal-ebm-harness**	How it works, grounded in the actual code

#	Document	What it answers
-	**THESIS-collective-nonparametric-intelligence**	The intellectual bet + the honest wall (foundation)
-	**ECONOMICS-cooperative-commons**	The token, the commons, the reward economics
03	**ONCHAIN-privacy-solana**	Weights on-chain, the frecency cache, the Solana moat, privacy & ZK
04	**PROOF-traction**	What is built and *proven*, with test evidence
05	**RISK-honest-wall**	The risk register - where this breaks and what must be true
06	**DUE-DILIGENCE-FAQ**	Direct answers to the diligence questions

Start at 01 for the product, 02 for the tech, 04 for the proof, 05 for the risks.

Status legend

Every substantive claim in this dossier is tagged:

- **BUILT+PROVEN** - code exists on the `arc-lean` branch with tests that pass in the build gate; verifiable by reading the source and running `zig build test`.
- **DESIGNED** - specified in detail, mechanism worked out, not yet fully built or not yet exercised end-to-end on production data.
- **RESEARCH-DIRECTION** - a bet or an open problem; plausible, motivated, but unproven at this scale. This is where the honest wall lives.

If a claim is not tagged, treat it as narrative framing, not a technical assertion.

Where the company is, honestly

One founder, deep in a two-to-three-month build. The harness is real and running locally; it has one-shotted tasks most agents cannot (see 04). It is **not yet** publicly released **- deliberately: the energy-based design is power-hungry and is** being contained, and the on-chain layer is being hardened. The pivot from the original Unbrowse (a commons for agents to browse the internet) to Aiko (an agent that "unbrowses" your *own computer's context*) is recent and partial; Unbrowse still exists and is the sister project this inverts. This dossier documents the Aiko direction as it stands today, with the seams visible.

*Confidential DD dossier. Status tags: **BUILT+PROVEN / DESIGNED / RESEARCH-DIRECTION** (see `00-README-DUE-DILIGENCE.md`).*

The product in one line

If you can do it on your laptop, ask Aiko. Aiko is a local agent that does the things you'd do - reading your messages, understanding your context, driving your applications - so you don't have to.

Who it's for

The core customer is **anyone who wants a personal agent that truly understands** their context **- the existing user of ChatGPT / Claude / Claude Code who has** tried an open agentic tool ("open-*") and found it unusable, unreliable, or unsafe on real work. Aiko is a **better open agent for computer use**: same ambition, but reliable enough and private enough to actually run on your machine, over your real data.

What it feels like (the use cases)

- **Inbox triage in your voice.** Coworkers are bombing you on Telegram. Aiko reads the thread, tells you *what actually applies to you*, and drafts a reply *in your voice* - grounded in how you've replied before.
- **Real-world tasks, end to end.** It's your mum's birthday. "Find a restaurant nearby and book a table for six at 6:30." Aiko finds it, checks availability, and books it - a multi-step task across apps and the web.
- **Anything on the laptop.** The product surface is not a chat box; it is *your computer*. Aiko calls the right binaries and drives the right apps at the right time - less like a chatbot, more like reflexes: a muscle that contracts and fires the correct action for the moment.

Why it beats a raw frontier LLM for computer use

A frontier LLM driving a computer is a brilliant generalist with no memory of what worked here before and no structure forcing it to be rigorous. Aiko wraps the model in a **harness that forces first-principles decomposition**: every task is broken into atomic sub-problems, and the model is made to reason mechanically - *which layer owns this step, where does the problem actually live, what fixes it* - rather than free-associating. The harness is the substrate; the model is one component inside it.

The proof point (BUILT+PROVEN, demonstrated locally): the harness - running Opus as the reasoner inside it - **rebuilt a Wine compatibility layer to launch** and run Overwatch on a Mac in effectively one shot. **Not *playing* the game -** getting a competitive Windows title to launch on macOS from a standing start. Most agents cannot do this; you cannot simply tell a general agent "make Overwatch run on my Mac" and have it work. It worked because the harness forced the model to solve the routing/travel problem one layer at a time, from first principles, and to locate and fix the actual failure - not because the base model got luckier. (See `04-PROOF-traction.md`.)

The two halves: harness + a small grounded model

Aiko has two parts, and the distinction matters for diligence:

1. **The harness is the Energy-Based Model.** It is a *recommendation/routing* system that learns which action to fire next from the collective history of what resolved before (see `02-ARCHITECTURE`). This is the durable, proprietary asset - **BUILT+PROVEN** as a router.

2. **A small, non-parametric local model, grounded by the harness.** Separately, a compact instruction-following/reasoning model is being trained to be as *small* as possible so it runs locally. It "doesn't know much" on its own - but because it is **grounded** by the harness (which can feed it live, retrieved knowledge about how the world and the machine work), it punches far above its size: smarter than the average model of the same footprint. Grounding, not parameter count, is where the capability comes from. **Status: the small model** is DESIGNED / in-training; the grounding mechanism (retrieval + energy routing) is BUILT+PROVEN.

The headline pitch - *"twice the reliability, half the cost"* - is the **design** target **of this combination (a small grounded model routed by a cheap energy** harness). It is not yet an independently benchmarked result; see the honest wall in `05-RISK-honest-wall.md`.

What "learn as it works" means

Aiko learns **on-fire**: every time it declares an intent, satisfies it, or fails, that verdict folds into the harness's prior immediately - no training run, no labels, no pipeline. Satisfied = good, died = retry. The system's own experience *is* the training signal, and it is realtime: a new fact indexed or a new verdict fired sharpens the next decision now, not after the next retrain. **BUILT+PROVEN** (train-on-fire fold; energy-ranked resolution).

Product status, plainly

Aiko is **used by the founder daily but not yet publicly released.** Two reasons, both honest: (1) the energy-based design is power-hungry and is being contained so it's safe and cheap for others to run; (2) the on-chain layer (which is how the weights become shareable and the storage cost is offloaded) is being hardened. The capability is real and demonstrated; the packaging is in progress.

*Confidential DD dossier. Status tags: **BUILT+PROVEN / DESIGNED / RESEARCH-DIRECTION**. Everything below is grounded in the `arc-lean` source tree; file names are real and readable. Foundation: `THESIS-collective-nonparametric-intelligence.md`.*

The shape of the thing

Aiko is a single native binary (Zig, zero external dependencies) that is at once: a **contract substrate** (an append-only ledger of intents and verdicts), an **energy-based harness** (a non-parametric router over that ledger), and a **client** to a live index of the world (Unbrowse) and to Solana. There is no Python, no model server, no data files: the embedding is baked in, the "weights" are reconstructible from the ledger, and the reasoning model is grounded through the harness rather than baked into it.

1. The atom - a contract neuron

The unit of everything is a **contract neuron**: one append-only JSONL row recording an intent (a *declare*) and its lifecycle (satisfied / died / purged - the terminal alphabet is exactly those three). A neuron carries its action, its lineage signature (every row is signed from a single wallet-derived root key - `lineage.zig`), and its pointer type. Contracts compose into a graph via `parent_id` containment and `contract:`-ref cross-edges. **BUILT+PROVEN** (`contract.zig`, `ledger.zig`, `pointer.zig`, `terminal.zig`).

The ledger is the shared, auditable memory. It is not a log *of* the system - it *is* the system's state, and the model is reconstructed from it.

2. The energy model - non-parametric, ~1 KB, trained on-fire

The harness's model is deliberately **not** a big parametric net. It is two thin, cheap primitives:

- **Embedding (`embed.zig`)** - a deterministic feature-hashing embedding (the hashing trick): each token and character-trigram hashes into a fixed-dimension, L2-normalized vector. Baked into the binary - zero data files, zero network. Cosine over these vectors captures lexical + sub-word overlap. **BUILT+PROVEN.**
- **Energy prototype (`ebm.zig`)** - a `Prototype` is a **reward-weighted** running-mean centroid **over context vectors (West 1979; O(1) per fire).** `foldText(action, reward)` folds a firing in: positive reward pulls the centroid toward it (Hebbian potentiation), negative pushes away (inhibition - the contrastive negative phase). The **energy** of a context is its negative cosine to the prototype; routing is **argmin energy**. **BUILT+PROVEN.**

Each such head is ~1 KB and reconstructible from the ledger. **The knowledge is** not in the weights **- it is in the retrievable index (the world) plus the** collective firing history (the ledger). A thought costs a retrieval + a centroid fold, not a forward pass through a giant net.

3. Fractal keys - multi-scale priors for free

The same firing folds into the *same atom* at many scales at once - `invoke:{event}`, `:L{layer}`, `:dim:{w6h}`, `:verb:`, `:ptype:`, `:root`. One cheap operation yields a **coarse-to-fine router**: the collective can prioritize at every granularity - per-event, per-layer, per-dimension - **without a separate**

Aiken Confidential - Prepared for investor due diligence. Not for distribution.

model per scale. **This is the "fractal" in the name: the EBM is not one model but** many nested thin ones, each wrapping a different facet of a route, broken down to first principles. **BUILT/DESIGNED** (fractal key convention in `fractal.zig` / `pattern.zig`; the arc-lean router exercises the per-run centroid directly).

4. Energy-ranked resolution - the router IS the harness

This is the load-bearing mechanism and the falsifiable atom of the whole thesis. When the substrate has a frontier of candidate contracts to fire next, it does **not** scan them in arbitrary order. `energyRankCandidates` (in `main.zig`) reconstructs a `Prototype` from the run's resolved history (each satisfied action folded with reward +1, each died with -1 - the train-on-fire signal, the ledger *is* the weights), then **sorts the frontier best-first by energy** so the most "resolved-like" candidates fire earliest - shorter cascades, less wasted work.

It is **always on** and **reorder-only**: every candidate still fires, so nothing is starved (a fairness floor holds by construction). It is the **thinking-budget** allocator **- it spends scarce compute/spend where history says it pays. BUILT+PROVEN**, and *measured*: on the prove-or-kill benchmark it settles a producer/consumer graph in **2 passes vs 3** for scan-order, mutation-verified (reversing the sort restores 3 - so the ranking, not luck, is doing the work). This is the atom "the collective routes its own thinking," proven before the cathedral. (See `04-PROOF-traction.md`, `PLAN-contracts-as-onchain-ebm-weights.md`.)

5. Ledger-reconstructible weights - the model is the commons

Because every weight is a fold of ledger rows, **anyone can rebuild the prior from** the shared ledger **(`replayFromLedger`, on `main`). The "weights" are the** collective experience, in the open - forkable, inspectable, re-derivable. No single node is "the model": the graph of contracts + the ledger-reconstructed EBM + the indexed world is the mind. This is also what makes the weights **portable** on-chain**: a weight is just content-addressed ledger data (see** `03-ONCHAIN-privacy-solana.md`, and `cas.zig` - content-addressed store, dedup-for-free, **BUILT+PROVEN**).

6. Grounding a small model - capability without parameters

The reasoning model is trained to be **as small as possible** (non-parametric in spirit: it holds instruction-following and reasoning, not world-knowledge). On its own it "knows little." But the harness **grounds** it: retrieval brings the relevant slice of the world and the machine to bear, and energy routing tells it which action history says works. A small model grounded this way is **smarter than** the average model of the same footprint **- capability comes from grounding, not** scale. **Status: the small model is DESIGNED / in-training; the grounding** mechanism (retrieval + `embed`/`ebm` energy routing) is BUILT+PROVEN. **The leap** from "thin router" to "reasoning-grade intelligence" is the honest open problem - stated plainly in the THESIS and `05-RISK-honest-wall.md`.

7. The module map (what's real, by concern)

Concern	Modules (`libcontract/src/`)	Status
Energy harness	`embed`, `ebm`, `energy_route`, `pattern`, `fractal` + the `main.zig` router	**BUILT+PROVEN** (router measured)
Contract substrate	`contract`, `ledger`, `pointer`, `terminal`, `cache`, `guards`, `capability`	**BUILT+PROVEN**
Identity / truth-root	`lineage`, `keystore`, `bible`, `verses`	**BUILT+PROVEN**

Concern	Modules (`libcontract/src/`)	Status
Commons economics	`cas`, `reward`, `usage`, `usage_ledger`, `epoch`	**BUILT+PROVEN** (local)
Solana / on-chain	`voltr`, `stake`, `payout`, `broadcast`, `broadcast_card`, `solana_rpc`, `solana_tx`, `base58`	**BUILT+PROVEN** (read/derive live; write = operator-gated)
Small reasoning model	(separate training track)	**DESIGNED / in-training**

Every module ships its tests *inside its source file*, exercised by `zig build test` and gated by `scripts/build.sh` (unit tests + 53 doctrine-parity claims + a signed self-attestation). Green build = every claim above has matching, tested code.

THESI
Collec
Intelli

The bet, in one sentence

Intelligence is **retrieval + routing + verification over a complete, live index** of the world **- not weights. Scale it with *coverage* and *collective** experience*, not parameters; and the marginal cost of a thought falls toward the cost of a lookup and a fold.

This is the opposite scaling law from frozen parametric LLMs, and it's what makes the four properties the user named - **lean, private, realtime, collective** - fall out of the architecture rather than being bolted on.

Why each property follows from the design

Lean / low-compute - because knowledge lives in the index, not the weights

The substrate's EBM is **non-parametric**: per-key centroids (`ebm.zig`, `E = -cosine(x, mean)`) and thin linear energy heads (`jepa.zig`, `||Wx-y||2`), each ~1 KB, **reconstructible from the ledger** (`replayFromLedger`). The "knowledge" is not baked into billions of parameters - it's the *retrievable index* (the world, via unbrowse) plus the *collective firing history* (the ledger). A thought costs a retrieval + a centroid fold, not a forward pass through a giant net. The model doesn't *store* the world; it *addresses* it.

Always realtime - because the index updates now and the model folds on-fire

A parametric model is stale the instant its training cutoff passes. This system has no cutoff: **unbrowse indexes the live internet**, and the EBM **folds on** every declare/satisfy/die **(train-on-fire). The collective's beliefs reflect the** world *and its own experience* as of this moment. New fact indexed -> available immediately. New verdict fired -> the prior sharpens immediately. No retrain lag.

Private - because inference is local and you contribute verdicts, not data

Intelligence runs **locally** over your own data plus a shared index; your data stays sealed under your key (envelope encryption + the receptor grant, from the storage plan). You don't ship your data to train someone else's model - you fire locally and contribute **signed, capability-gated verdicts** to the commons. The raw stays yours; only *what worked* (attested, not the content) joins the shared prior.

Collective - because the ledger is shared memory and the graph is the mind

The ledger is a shared, append-only memory the whole swarm reads and reconstructs the EBM from. Every agent's terminal verdicts fold into the same reconstructible prior; the **"two or three witnesses" corroboration doctrine** keeps that prior honest. No single node is "the model" - the **graph of contracts + the shared** ledger-EBM + the indexed world **is the mind. Intelligence is a property of the** collective, the way a market or a science is.

How the fractal EBM harness specifically helps

- **Fractal keys -> multi-scale priors for free.** The *same* atom folds across

`invoke:{event}`, `:L{layer}`,`:dim:{won}`,`:verb:`,`:ptype:`,`:root`. One cheap operation yields a coarse-to-fine router - the collective can prioritize at every granularity without a separate model per scale.

- **Train-on-fire -> the system's own history is the training signal.** Satisfied = good, died = retry. No labels, no pipeline, no training run - the substrate learns from doing.
- **Ledger-reconstructible weights -> the model is the commons, portable and** auditable. **Anyone can rebuild the prior from the shared ledger; it can be** forked, inspected, and re-derived. The "weights" *are* the collective experience, in the open.
- **Energy-ranked resolution -> a thinking-budget allocator.** It routes scarce compute / spend to where history says it pays (the plan's Phase 1). This is the mechanism by which a low-compute system stays low-compute *as it scales*: it spends attention where the energy gradient is steepest.

The compounding loop (why it could grow toward "smarter than its parts")

- **Parametric AI:** intelligence prop. params x data x compute - static, centralized, and your data trains their model.
- **This:** intelligence prop. *index coverage* x *collective firing history* x *routing/verification quality* - live, distributed, and your data stays yours.
- As unbrowse indexes more of the world and more agents fire and corroborate, two things widen together: the *retrievable context* (what can be brought to bear) and the *sharpness of the prior* (what history says works). Each is cheap per step and never goes stale. A system whose knowledge and whose judgment both compound with use, at near-zero marginal cost, is a genuine candidate for exceeding any individual part.

THE HONEST WALL - what must be true, and where this breaks

(In the substrate's own tradition: state the limits as loudly as the vision, or it's marketing.)

1. **The current energy model is shallow - a router prior, not a mind.** A cosine-to-one-centroid and a linear `W` are soft priors over "what resolved before." Real reasoning is *not* in this model today. The leap from "thin router" to "intelligence" is unbuilt, and it comes from one of two places, both unproven at this scale: richer *composed* energy models (compute cost returns), or *emergent* collective composition (that many thin routers + verification compose into reasoning). Do not mistake the scaffold for the building.

2. **Retrieval != reasoning.** Indexing the whole internet gives *knowledge access*, not thought. Superintelligence needs composition, planning, and verification *over* retrieved knowledge. The energy-descent-as-generation primitive (`jepa.plan`) is the seed of this, but it's linear and shallow. The reasoning layer is the hard, unsolved core.

3. **"Low compute" is low *marginal*, not low *total*.** Embedding and serving retrieval over the live web at scale is real, heavy infrastructure. The compute *moves* from training to indexing/serving - it does not vanish. The honest claim is cheap-per-thought, not cheap-overall.

4. **Verification is the crux, and the single biggest risk.** A collective that learns on-fire from verdicts is only as good as its verdicts. Bad or gamed

verdicts don't just add noise - they *train the shared prior toward delusion*. The truth-root ("thy word is truth") + corroboration ("two or three witnesses") is the designed guard, and it is load-bearing: if it fails, the collective gets confidently, collectively wrong. This is the difference between a wisdom-of-crowds and a madness-of-crowds, and it is not automatic.

5. **Query privacy over a shared index is unsolved by encryption-at-rest.** Your data-at-rest is private (sealed). But *querying* a shared index can leak what you're asking. True query privacy needs PIR / local replicas / ZK - a separate, hard problem. "Private" holds for your data; not yet for your attention.

6. **"Superintelligence" is a bet, not a result.** Collective systems - markets, evolution, science - *do* exceed any participant, so it isn't crazy. But whether *this* reaches "super," and not just "useful," is untested and hinges entirely on #1, #2, and #4 holding. Claim it as a *falsifiable direction*, not a destination.

The falsifiable first step (this is the discipline)

The entire thesis has a smallest testable version, and it's already the plan's **Phase 1**: does energy-ranked resolution *measurably* beat scan-order at choosing what fires next? That single question is the atom of "the collective routes its own thinking." **If the thin version doesn't help, the grand version** won't **- and we'll have learned it for the cost of a benchmark instead of a** cathedral. Prove the atom before believing the collective. Everything above earns its right to be built one measured gate at a time.

ECON
Ranke

A governed common-pool resource secured by bonded accountability. Unifies the ranked-energy-contract commons with the Unbrowse Maintenance Network (Proof of Indexing + Bonded Accountability, Tham 2026): a ranked energy contract IS a route, so the same economics apply without reinvention.

> *And all that believed were together, and had all things common ... and parted them to all men, as every man had need. ... Neither was there any among them that lacked.* (Acts 2:44-45; 4:34)*

Companion to `THESIS-collective-nonparametric-intelligence.md` (why a lean non-parametric collective can be intelligent) and `PLAN-contracts-as-onchain-ebm-weights.md` (how it's built, prove-the-atom-first).

0. The unifying insight

A **ranked energy contract** - a reusable, ledger-reconstructible prior that resolves some context - is the same *kind* of object as a **route** in the maintenance-network paper: a shared, reusable asset that **decays** (the world drifts; the prior goes stale) and whose **freshness/quality is a non-rival public good. So the paper's economics transfer 1:1. This document adopts them.**

Naming it correctly (per Ostrom): the graph of ranked contracts is a **common-pool resource** - hard to exclude, but *depletable through uncoordinated use*. The depletion is **epistemic**: every agent that leans on a stale prior and propagates it as correct lowers the trustworthiness of the graph for everyone. A prior that is *wrong with confidence* is worse than none.

1. Fund freshness by paying for it - reward, don't punish

Freshness of the graph is a **non-rival, near-non-excludable public good** -> a bare usage fee (pay-per-call, revenue to nobody) **structurally under-provides** it: everyone free-rides, nobody is paid to keep a prior fresh, priors rot.

The textbook Pigovian fix prices the *harmful act* - a **slashable bond** that seizes stake on a false freshness claim. It works, but it is **punitive, requires** valuing the harm, invites adversarial slashing disputes, and raises the bar to participate**. There is a positive-sum instrument that internalises the SAME** externality by pricing the *good* instead of the harm:

> ***Redirect the usage fees to the people who keep the used routes fresh.***
> *Fees are paid out **in the same asset they arrive in - USDC** (no conversion) - to the stakers whose routes were actually **used** and still **verify**, weighted by the usage they generated.*

Now freshness is *funded by the very demand it serves*: a route people use earns its maintainer real yield, and a route that rots simply stops earning. The "slash" is transmuted into **redistribution** - a route that fails re-verification forfeits its accrued fees, and those fees flow to the **honest, verified** maintainers (`reward.zig`, built + proven: an unverified route earns 0, its fee makes the honest earn *more* than their own usage - punishment became provision). **Luke 10:7** - "the labourer is worthy of his hire." **Nobody's principal is seized; nothing is** burned; the users' payments still create value by rewarding whoever is correct.

2. Two assets, deliberately separate

Rail	Asset	Job	State
Usage -> reward	USDC over `x402` (paid out **as USDC - no swap**)	meter calls *and* pay the USDC out to used+verified maintainers (`reward.zig`)	usage rail exists; distribution built (local), on-chain payout to wire
Stake	**FDRY** (base token)	**entry to the earning tier + Sybil cost**; bonded in the Voltr vault (`voltr.zig`); **never seized**	staking built (arc-lean)

They stay separate on purpose: **reward in USDC (real external revenue, already** the inbound asset), stake in FDRY (skin-in-game). **Paying rewards in FDRY would** be circular/inflationary; paying in USDC means staking FDRY earns *real* yield, so **FDRY's value grows with real usage of a trustworthy graph - without inflating** FDRY**. The reward asset need only be *real and non-minted*; USDC beats SOL for it** (stable, already flowing in, zero conversion, no FX risk on the maintainer). FDRY is the security/eligibility asset, not the reward. Its **fair distribution is a** security parameter, not marketing**: a coordination asset launched for money** cannot become trustworthy.

3. Proof of re-verification - freshness as a checkable artifact

A bond only punishes a false claim *after* it's found false - it presupposes a way to find it false. So freshness is made a **verifiable artifact**, the ranked-contract analog of the paper's **Proof of Indexing**:

- A maintainer **re-verifies** a contract - re-runs it against fresh evidence and attests that it still resolves and its observed shape matches - and commits a **content-addressed, hash-chained attestation** to the ledger. **`cas.zig`** (built) is exactly this substrate**: content-addressed, integrity-checked,** independently reproducible. Anyone can re-verify and check they reconstruct the same attestation. Freshness stops being say-so and becomes a checkable object.
- One proof certifies a *moment*; **repeated proofs on a cadence** (set by the trust tier) are what show a prior is *kept* fresh - the Proof-of-Spacetime property, not the point one. `ebm.zig`'s 30-day freshness decay is the clock that says when re-verification is due.
- We attest *what* is observable (it resolves, the shape matches) and never publish *how* the capture/re-verification is produced - the private surface stays private; the proof is public.

4. Trust tiers (Ostrom's nested enterprises) - and why it's "optional"

Not all traffic needs the same assurance; pretending it does taxes a harmless price-check at the same rate as a funds-moving call.

- **Open tier - the socialized commons.** Low-risk, idempotent priors. **No bond,** best-effort, free**, and honest about being best-effort. This is the ledger-reconstructible public good**: anyone rebuilds the shared centroids from the public ledger for free - **un-encloseable by construction**. The free, fast majority.
- **Bonded / high-trust tier - the optional premium.** Authenticated, high-value, or must-not-be-wrong priors. **FDRY-bonded, proof-of-re-verification-backed,** slashable **by a threshold quorum. A maintainer stakes FDRY behind "this prior** is fresh/good"; if independent re-verification refutes it, the bond is slashed.

"Own it via Voltr, but optional" fits exactly: you run a **maintainer/bonding** vault **serving the high-trust tier; the open tier stays free and un-encloseable.**

You are a *maintainer*, not an enclosee. Staking is opt-in — it buys you *into* the verified tier; it never gates the commons.

5. "Simulate socialism" = Ostrom CPR governance, done the empirical way

Durable commons are **neither privatised nor centrally administered - governed by** the appropriators **(Ostrom's Nobel finding). Three principles map directly:**

- **Clearly-defined boundaries** - who may claim a prior is fresh, and who may draw on it (the receptor grant + the trust tiers).
- **Graduated response** - *reward-withholding* tiers: a first low-confidence miss dampens a route's earnings; a repeated or high-value false claim cuts them to zero and redistributes them. No principal is ever seized - the sanction is denied reward, not destroyed capital.
- **Nested enterprises** - the trust tiers themselves: lightly-governed open routes inside tightly-governed high-trust ones.

From each according to ability (everyone's verdicts train the base prior) -> *to each according to need* (free access to the commons prior). Only the **surplus** - bonded verification, priority compute, private priors - is ever paid. That is not utopian socialism; it is a **governed commons**, the thing that actually works.

6. THE HONEST WALL - what's live, what's reference, what breaks

(As loud as the design, per the substrate's own tradition - and the paper's.)

1. **Formal Sybil resistance is NOT closed, and reward-withholding is softer than** slashing. **Denying future reward is a weaker deterrent than seizing stake - a** spammer loses nothing they held. The stake-to-enter requirement is the Sybil cost that remains (required, never seized); it raises the bar but does not formally solve Sybil. A conscious trade: more open and non-punitive, less costly to attack, than a slash.
2. **Usage-gaming: addressed structurally, not formally closed.** Because payout prop. usage, a maintainer wash-calling their own routes to farm the payout is the attack that replaces "confidently-wrong data." `usage.zig` (built + proven) removes the *profit motive* with two rules: **self-exclusion** (a payment from the route's own maintainer adds zero weight but still enters the pool - a self-dealer just donates to the honest) and a **per-payer cap** (weight tracks breadth of distinct demand, so faking weight W needs ~ W/cap distinct funded wallets, each paying real fees). End-to-end test: a wash-trader who pays 1000 to its own route earns **0** and the honest maintainer takes the whole pool. **What remains open**: this bounds but does not *formally solve* Sybil - an attacker with many cheap distinct wallets can still buy breadth; the per-payer cap + the real x402 fee per wallet are the cost floor, not a proof. Identity cost (stake-weighted payer reputation, proof-of-personhood) is the next lever if the cost floor proves too low in practice.
2. **The token's fair distribution is load-bearing.** FDRY exists strictly for the security function. If its distribution is a money-grab, the whole trust model collapses - "a coordination asset launched for money cannot become trustworthy."
3. **Yield on the bond couples the security asset to market risk.** Holding the FDRY bond in a Voltr yield vault is attractive (idle collateral works), but the asset that *secures* the graph should track *network integrity*; a strategy loss that impairs the bond weakens the security it's supposed to provide. A conscious tradeoff, not a free lunch.
4. **Reputation is now a checkable artifact - but re-verification quality is the** new crux. **Proof-of-re-verification dissolves "reputation != correctness" only**

as far as the re-verification is honest and independently reproducible. Garbage re-verification -> confidently-wrong "fresh" priors. The quorum + graduated slashing is the guard, and it is the same guard the whole collective bet rests on.

5. **On-chain only for the genuine commons.** All of this is worth its cost ONLY when the trustless, host-less, shared case is real. Single user or trusted group -> skip the chain; the energy router + encrypted local/Arweave store already deliver lean/private/realtime at zero SOL.

6. **Live vs reference vs design.** Be explicit (as the paper is): the route/contract resolution + the `x402` usage rail are live; the energy router, `cas.zig` attestation substrate, and receptor are built+proven here; the bond/challenge/slash economics are *design + reference*, not a deployed production mechanism.

7. Mapping to substrate primitives

Mechanism	Primitive	State
Socialized base prior	ledger + `ebm_store.replayFromLedger`	exists (main)
Energy ranking (the quality signal)	`ebm.zig` + Phase-1 router	**built + proven (arc-lean)**
Proof-of-re-verification substrate	`cas.zig` content-addressed attestation	**built (arc-lean)**
Freshness cadence / demurrage	`ebm.freshness` (30-day half-life)	exists (main)
Access boundary	receptor grants (`capability.zig` F1)	**built (arc-lean)**
Usage rail	`x402:` (USDC)	exists (main)
Solana read / RPC (blockhash, balance, airdrop)	`solana_rpc` + `base58`	**devnet-proven (arc-lean); zero-dependency (std-only, native TLS - no curl/shell), env-free**
Trustless settlement + anchoring (write)	`iq_ledger` / `solana_tx` / `agency`	exists (main); deferred until on-chain purpose built + env-config refactored to param/ledger
Corroboration / quorum for slashing	"two/three witnesses" doctrine	exists (doctrine)
FDRY stake (entry + Sybil cost)	`voltr.zig` - native Voltr deposit -> stFDRY receipt	**built + proven (arc-lean); real mainnet vault, PDA derivation matches on-chain authority; operator-gated card, zero-dependency**
Live stake read (stake field <- chain)	`stake.zig` - a wallet's stFDRY balance = its bond, read via `solana_rpc`	**built + proven (arc-lean); live mainnet read verified (derive ATA -> getAccountInfo -> parse amount); a read, no operator gate**
Sybil-dampened usage accounting	`usage.zig` - self-exclusion + per-payer cap -> dampened weight	**built + proven (arc-lean); wash-trade-is-a-donation tested end-to-end**
Usage -> USDC reward distribution (replaces slash)	`reward.zig` - weight-driven, verification-gated, stake-gated payout	**built + proven (arc-lean, local); conservation + forfeit->honest properties tested**
Re-verification verdict	`cas.reproduces` - fresh re-run reproduces the committed attestation (a hash equality, not a trust assertion)	**built + proven (arc-lean)**
Epoch harness (the whole loop)	`epoch.runEpoch` - re-verify -> usage -> reward, in one pass	**built + proven (arc-lean); capstone test drives every gate in one epoch**
Ledger <-> usage events	`usage_ledger` - reads the REAL `flex_settle` settlement rows (already emitted live by `appendFlexSettleRow` on `flex:settle`) + a clean `usage` shape; JSONL->`usage.Event`	**built + proven (arc-lean); live emit already exists - parser wired to it; full ledger-line->payout round trip tested**
Route -> owner identity	`usage_ledger.resolveOwners` - a route's `declared` row -> `env_pubkey` (its owner), hashed to the same id space as payers	**built + proven (arc-lean); owner sourced from the ledger drives self-exclusion end-to-end**

Mechanism	Primitive	State
Route -> committed attestation	`usage_ledger.resolveAttestations` + `emitAttestRow` - the `attest` row convention; latest wins	**built + proven (arc-lean); drift detected from the attest row + fresh re-run**
Fully-live assembler	`usage_ledger.runEpochFromLedger` - owner + attestation + usage FROM THE LEDGER, stakes injected live	**built + proven (arc-lean); whole epoch from ledger text + live stakes, every gate ledger-sourced**
On-chain payout	`payout.zig` - SPL `TransferChecked` USDC card per maintainer (no swap), operator-gated	**built + proven (arc-lean); batch card from an epoch's payouts; canonical USDC mint, emits - never sends**
Operator submit path	`broadcast.zig` - sign (`solana_tx`) + send (`solana_rpc`); the only module that moves value	**built + proven (arc-lean); wire is cluster-valid (devnet: dummy blockhash -> BlockhashNotFound, i.e. deserialized + simulated); gated on an operator keypair, never called implicitly**
Card -> instruction bridge	`broadcast_card.zig` - parse an emitted card + apply ix flag template (deposit_vault / transfer_checked)	**built + proven (arc-lean); parses the REAL payout + deposit cards; refuses an unknown ix (no guessed flags)**
Operator broadcast tool	`broadcast-tool` - standalone binary: `<keypair.json> <card.json> <network>` -> signature	**built + proven (arc-lean); end-to-end (load key -> parse -> sign -> submit) reaches the chain. OUTSIDE the substrate's declare/redaction flow - a money-move can't be a redacted, no-verb public action; it is the operator's deliberate run**

8. The falsifiable first step (discipline)

Before any FDRY or on-chain settlement, the smallest testable claim, **all local**: a **re-verify -> attest -> bond -> slash-on-refutation** flow - re-run a ranked contract, commit a content-addressed attestation via `cas.zig`, post a (mock) bond behind the freshness claim, and have an independent re-verification under a quorum **slash** the bond when the attestation fails to reproduce. If the *local* bond-and-slash is correct and un-gameable under corroboration, then - and only then - settle FDRY on-chain. Prove the economics off the chain before paying the chain to enforce them, exactly as we proved the router and the dedup first.

Bottom line: the chain is not a database; it is the **trust layer of a governed** commons**. FDRY's one job is to make "whose freshness claim is real, who** maintained it, who gets slashed" un-fakeable and host-less - so a collective can share and *keep fresh* a graph of ranked priors without a platform enclosing the value or letting it rot. That is the reason the on-chain layer exists, and the only reason it is worth its cost.

*Confidential DD dossier. Status tags: **BUILT+PROVEN / DESIGNED / RESEARCH-DIRECTION**. Companion to `ECONOMICS-cooperative-commons.md` (the token + reward economics).*

Why on-chain at all

Aiko's energy-based design has one honest cost: **it grows storage as it learns.** Every action trains a prototype and folds new history; run it long enough on one laptop and the weights + ledger accrue. The design turns that liability into a feature by **putting the weights on-chain**:

- **Weights become shareable.** A weight is just content-addressed ledger data (`cas.zig` - content-addressed store, dedup-for-free, integrity-checked; **BUILT+PROVEN**). Anyone with access can reconstruct the same prior from the same rows. You can *call your AI* - hand it to another device, fork it, audit it.
- **Storage cost is offloaded.** The machine holds a **frecency cache** - hot, frequently/recently used weights persist locally; cold ones live on-chain and are pulled on demand. Store **primarily on-chain**, cache locally what's in frequent use. **DESIGNED** (the caching policy; `cache.zig` is the local layer; the on-chain tier uses the anchoring path below).
- **Shared data stays cheap.** Content-addressing means a weight referenced by many contracts is stored (and paid for) **once** - the dedup is a property of the addressing, not a special case. **BUILT+PROVEN** (`cas.zig`).

Status of full on-chain weight storage: DESIGNED, partially proven. The primitives are real - content addressing (`cas`), live Solana reads (`solana_rpc`, devnet-proven), on-chain hash anchoring (`iq_ledger` on `main`, anchors claim hashes to a deployed Solana program), and a rent model that bounds what goes on-chain (<= ~4 KB on-chain, larger = commitment-only + off-chain). The end-to-end "weights live on-chain, laptop is a cache" loop is assembled from these but not yet running in production at scale.

The Solana moat

The strategic bet is distribution + storage, stated plainly:

> *Once the EBM approach is shown to work, frontier labs *will* copy it - it's proprietary tech, but it's copyable. **The moat is not the algorithm; it's the** ledger. **No one in AI has a distributed ledger the size of Solana. A ledger** that can hold all the weights in the world - and a distribution channel already in millions of wallets - is the durable asset. Win the distribution and the storage substrate before the technology is commoditized.*

And the economics close on themselves: **increased Solana transaction volume** means the product pays for itself. **Every weight write, every attestation, every** settlement is on-chain activity; as usage grows, the on-chain volume it generates funds the substrate it runs on. (The full reward economics - usage fees paid out to stakers in USDC, FDRY as the security stake, reward-not-slash - are in `ECONOMICS-cooperative-commons.md`, **BUILT+PROVEN locally**, with the real mainnet FDRY vault integration in `voltr.zig`, **BUILT+PROVEN**.)

Honest status of the moat: this is a *go-to-market and defensibility thesis*, not a technical lock-in that exists today. Solana gives distribution + cheap durable storage; it does not by itself prevent a well-funded lab from building the same harness on their own infrastructure. The moat compounds only if Aiko wins the network (the commons of shared, corroborated weights) first. See

Privacy - what holds, and what is a bet

The pillar is *"your information stays yours,"* and it decomposes into three distinct claims with three different maturities. Diligence should not conflate them:

1. **Local inference + sealed data - BUILT/DESIGNED.** Inference runs *on your machine* over your own data plus a shared index. You don't ship your data to train someone else's model; you fire locally and contribute **signed,** capability-gated verdicts **- attestations of *what worked*, not the content.** Your data at rest is sealed under your key (envelope encryption + the **receptor grant** access boundary - `capability.zig`'s opt-in receptor is **BUILT+PROVEN**: value is adopted only under a signed grant, measured at 100% cross-lineage enforcement).

2. **Zero-knowledge credential binding - RESEARCH-DIRECTION (not built).** The pitch says "ZK-proof credential binding creates privacy you can verify." Be precise for diligence: **there is no production zero-knowledge system today.** The current on-chain identity/attestation layer uses hash commitments and signed lineage, *not* a ZK proof system. Real ZK credential binding (e.g. Pedersen/Ristretto commitments + Schnorr + KVAC, avoiding SNARK trusted setup) is designed on paper and is the intended path - but it is unbuilt. Treat "ZK" as a **direction**, not a shipped feature.

3. **Query privacy over a shared index - UNSOLVED.** Your data *at rest* is private. But *querying* a shared index can leak what you're asking. True query privacy needs PIR / local replicas / ZK - a separate, hard problem, not solved by encryption-at-rest. **"Private" holds for your data; not yet for your** attention. **(Stated identically in the foundation THESIS - this is not new** caution, it's the standing honest wall.)

The one-line summary for diligence

On-chain is **real in its primitives** (content-addressing, live reads, anchoring, a real FDRY vault, the reward loop) and **a designed system** at the "weights fully live on-chain, laptop is a frecency cache" level. Privacy is **real for data-at-rest**, **a direction for ZK**, and **an open problem for query** privacy**. The moat is a** distribution/storage thesis**, compelling but not a** technical lock-in yet. Everything here is labelled so the seams are visible.

*Confidential DD dossier. Everything marked **PROVEN** is verifiable by reading the `arc-lean` source and running `zig build test` / `bash scripts/build.sh`. Founder-reported items awaiting independent replication are marked as such.*

The build gate (the honesty mechanism)

The whole substrate ships behind one gate: `scripts/build.sh` runs the unit test suite, **53 doctrine-parity claims** (mechanical checks that the code matches its own stated doctrine), a formatting pass, and a **wallet-signed self-attestation** of the binary. A green gate means every module's in-source tests pass *and* the code has not drifted from its documented invariants. This dossier's **PROVEN** tags are all downstream of that gate. It is currently green: *53/53 claims, signed.*

The falsifiable atom - energy-ranked resolution beats scan-order

The single most important proof, because the entire thesis rests on it: **does** the collective's energy prior actually route thinking better than arbitrary order?

- **Result: yes, measured.** On a prove-or-kill benchmark (producer/consumer contract graph), energy-ranked resolution settles the graph in **2 passes vs 3** for scan-order - fewer cascades, less wasted work - **16/16** runs.
- **Mutation-verified:** reversing the energy sort restores 3 passes. So the *ranking*, not luck or an unrelated change, is doing the work.
- **Non-breaking:** it is reorder-only; every candidate still fires (fairness floor holds). It is now **always on** - the default resolution path.

This is "the collective routes its own thinking," proven at the atom before building the cathedral. **PROVEN** (`main.zig` router; `examples/arc-agi3/bench-energy-router.sh`).

The product one-shot - Wine/Overwatch on macOS

The harness (with Opus as the reasoner inside it) **rebuilt a Wine compatibility** layer to launch and run Overwatch on a Mac, in effectively one shot **- a** competitive Windows title brought up on macOS from a standing start. This is the capability proof: most agents cannot do this, and you cannot get there by telling a general agent "make Overwatch run on my Mac." It worked because the harness forced first-principles, layer-by-layer decomposition and fault localization. **Founder-demonstrated** (a Loom/recording is the artifact to request for diligence).

ARC-AGI benchmark - reasoning without a large model

The founder tested an earlier version against **ARC-AGI-3** (a benchmark designed so that passing normally requires a large model) and **reached a high score** without using any large model **on the order of Grok/Opus - the point being that** the *harness*, not raw parameter count, carried the reasoning. **Founder-reported;** awaiting independent replication **(the arc-lean tree is mid-rebuild toward that** earlier capability - `examples/arc-agi3/`). Diligence should request the exact score, the task split, and a reproducible harness invocation.

The commons economics - proven locally, end to end

The cooperative-commons economics (`ECONOMICS-cooperative-commons.md`) are not a whitepaper sketch; the mechanism runs and is tested:

- **Reward, not slash.** Usage fees form a pool paid out to the maintainers whose routes were *used* and still *verify* - usage-weighted, verification-gated, stake-gated. Tested invariants: **conservation** (nothing burned), **forfeit -> value (a route that fails re-verification forfeits its fees to the honest, who** then earn more than their own usage), **stake-gated** (unstaked earns nothing). **PROVEN** (`reward.zig`).
- **Wash-trading is a donation, not a theft.** Sybil-dampened usage accounting (self-exclusion + per-payer cap): a maintainer wash-calling its own route earns **0** and the honest take the pool. **PROVEN** (`usage.zig`, `usage_ledger.zig`).
- **The whole loop runs from the real ledger.** `epoch.runEpoch` + `runEpochFromLedger` assemble stake (from chain), owner + attestation + usage (from ledger rows) and produce the payout set - a capstone test drives every gate at once from JSONL. **PROVEN.**

Real chain integration - reads live, writes are operator-gated

- **FDRY staking is real.** `voltr.zig` builds native Voltr vault instructions against the **real mainnet vault**; its PDA derivation **reproduces the actual** on-chain vault authority **from seeds - a wrong byte fails the build. Cross-** validated against Voltr's published CPI reference (program id, discriminators, account order, seeds all match). **PROVEN, without spending anything.**
- **Live reads work.** `solana_rpc` fetches live devnet blockhashes and reads a wallet's stFDRY stake from mainnet (`stake.zig`). **PROVEN.**
- **The submit path is cluster-valid.** `broadcast.zig` compiles + signs a legacy transaction; submitting it to devnet with a dummy blockhash returns `BlockhashNotFound` - i.e. the wire **deserialized and reached simulation** (a malformed wire errors "failed to deserialize"). So the serialization is byte-correct against the real Solana runtime. The operator utility `broadcast-tool` runs the full pipeline end-to-end (load key -> parse card -> sign -> submit). **PROVEN.**
- **Money moves only by the human's hand.** Every value-moving step emits a *card* and never sends; only the operator, running `broadcast-tool` with a funded key, fires it. This is a deliberate safety boundary, not a missing feature.

What "traction" honestly means today

- **Working software, daily use by the founder.** The harness is real and running.
- **Zero external users / not publicly released.** By choice (energy containment + on-chain hardening), not for lack of a build. This is pre-launch.
- **Reproducible technical proofs** (the router, the commons loop, the chain integration) are in-tree and gated.
- **Capability proofs** (Overwatch one-shot, ARC-AGI score) are founder-demonstrated and are the right things to ask for a recorded, reproducible demo of. The honest ask for diligence: a Loom of the harness one-shotting a fresh task, and the ARC-AGI run with its numbers.

*Confidential DD dossier. The substrate's own tradition: *state the limits as loudly as the vision, or it's marketing.* This is the section diligence should weight most. Nothing here is hidden elsewhere; it is collected for scrutiny.*

Technical risk

R1 - The energy model is a router, not a mind (the core bet). Today's model is a cosine-to-centroid plus a thin linear head - soft priors over "what resolved before." Real reasoning is **not** in this model. The leap from "thin router" to "intelligence" is *unbuilt* and comes from one of two unproven places: richer composed energy models (which return some of the compute cost) or emergent collective composition (many thin routers + verification composing into reasoning). **This is the whole company's central technical bet.** Mitigation: the falsifiable-atom discipline - prove each rung (the router *is* proven) before funding the next. Do not mistake the scaffold for the building.

R2 - Retrieval != reasoning. Indexing the world gives knowledge *access*, not thought. Superintelligence needs composition, planning, and verification *over* retrieved knowledge. The energy-descent-as-generation primitive is the seed, but it is linear and shallow. **The reasoning layer is the hard, unsolved core.**

R3 - Verification is the crux and the single biggest risk. A system that learns on-fire from verdicts is only as good as its verdicts. Bad or *gamed* verdicts don't just add noise - they train the shared prior toward delusion. The guard (a scripture-grounded truth-root + "two or three witnesses" corroboration + the reward economics' verification gate) is **load-bearing**: if it fails, the collective becomes confidently, collectively wrong. This is the line between wisdom-of-crowds and madness-of-crowds, and it is **not automatic.**

R4 - "Low compute" is low *marginal*, not low *total*. Embedding and serving retrieval over the live web is heavy infrastructure. The compute *moves* from training to indexing/serving; it does not vanish. Honest claim: cheap-per-thought, not cheap-overall. The "half the cost" pitch is a per-task target, not a total-infrastructure claim, and is **not yet independently benchmarked.**

R5 - Energy containment (the founder's own stated blocker). The design is power-hungry - it "leaks energy" by nature (it trains and grows storage on every fire). This is *why* it isn't released yet. The on-chain offload is the intended fix (weights on-chain, laptop as frecency cache), and is DESIGNED but not proven at scale. Until contained, the product is unsafe/uneconomical to hand to users.

R6 - Query privacy is unsolved; ZK is unbuilt. Data-at-rest is sealed, but querying a shared index can leak intent (needs PIR/local-replicas/ZK). And the headline "ZK credential binding" is a **direction, not a shipped feature** - today's identity layer is hash commitments + signed lineage. Marketing that says "privacy you can verify" is ahead of the code; diligence should hold the code's actual maturity (see `03-ONCHAIN-privacy-solana.md`).

Strategic / market risk

R7 - The algorithm is copyable; the moat is the network, not the code. If the EBM harness works, frontier labs can and will build it - it is proprietary but not un-reproducible. The defensibility thesis is **distribution + the ledger** (Solana's storage + wallet distribution + a commons of corroborated weights won first). That is a race, not a lock. If Aiko does not win the network before the

approach is commoditized, the moat does not hold.

R8 - Single-user, not yet a collective. The thesis's power (compounding intelligence, shared prior) requires *many* agents firing and corroborating. Today there is **one user** (the founder). Every "collective" property is architectural potential, not observed behavior. The network effects are unproven because the network does not yet exist.

R9 - Pre-launch, zero external validation. Not publicly released, no external users, no revenue. Capability is founder-demonstrated (Overwatch one-shot, ARC-AGI) and awaits independent, recorded replication.

R10 - A new-currency ambition adds scope and regulatory surface. The founder's end-vision includes defining a *new currency that rewards novelty/value-created* (beyond value-of-work). This is early and unformed, and paying token holders from usage introduces securities-like questions. It should be treated as a research aspiration, not a near-term deliverable, and de-scoped from the core investment thesis.

Execution / key-person risk

R11 - Single founder, deep in a rabbit hole. One person, ~2-3 months into a very deep build, self-aware about scope creep ("dug myself into a deep rabbit hole"). Enormous surface area (harness + model + on-chain + economics) for one person. Key-person risk is maximal; the first hire and a scoped roadmap are the obvious de-risking moves.

R12 - Two projects in superposition. Aiko is a partial pivot from Unbrowse; Unbrowse still exists. Focus risk: the pivot is not clean, and "which company is this" is a real question a first-round panel already flagged.

"Superintelligence" - a bet, not a result

Collective systems (markets, evolution, science) *do* exceed any participant, so the ambition is not crazy. But whether this reaches *super* - and not merely *useful* - is untested and hinges entirely on R1, R2, and R3 holding. It should be underwritten as a **falsifiable direction with a cheap first gate already passed** (the router), not as a destination being claimed.

The one honest sentence

The router is proven; the reasoning is a bet; the network doesn't exist yet; the privacy pillar is real for data and aspirational for ZK; and the moat is a race to own a commons before the idea is copied. Everything compelling about Aiko is downstream of bets that are named, cheap to test next, and being tested in order.

06 - D

Confidential DD dossier. Direct answers to the questions raised in the second-round diligence conversation. Cross-references point to the fuller treatment. Honest status tags throughout.

Q1 - What are you actually building? What's the core product and problem?

Product: Aiko - a local agent that does the things you'd do on your laptop, so you don't have to (triage your Telegram in your voice; find a restaurant and book the table; *if you can do it on your laptop, ask Aiko*). **Problem:** existing agentic tools for computer use are unreliable, expensive, and privacy-hostile - brilliant generalists with no memory of what worked here and no structure forcing rigor. **Core insight:** instead of "unbrowsing" the *internet* (the original Unbrowse), Aiko "unbrowses" the *context of your own computer* - building reflexes that call the right binary at the right time, from an agent that actually understands you. See `01-PRODUCT-aiko.md`.

Q2 - Who is the core customer?

Anyone who wants a **personal agent that truly understands their context** - the current ChatGPT/Claude/Claude-Code user who tried an open agentic tool and found it unusable. Aiko is a **better open agent for computer use.** See `01`.

Q3 - How is this different from what frontier labs ship?

Frontier labs are **parametric-LLM-centric** - intelligence prop. params x data x compute, static, and *your data trains their model*. Aiko is **harness-centric**: intelligence prop. index-coverage x collective-history x routing/verification, live and local, and *your data stays yours*. Very few players take **energy-based** models **seriously because the naive version needs a data-center to "simulate** reality." The breakthrough is **breaking the EBM into fractal atoms** - not one giant EBM, but many tiny ones, each wrapping a route from first principles - which makes it cheap and local. See `02-ARCHITECTURE`, foundation THESIS.

Q4 - Are people using it today?

The **founder uses it daily; it is not publicly released.** Two honest reasons: the energy-based design is power-hungry and is being contained (R5), and the on-chain layer is being hardened. Zero external users today - pre-launch by choice, not for lack of a working build. See `04-PROOF-traction.md`, `05-RISK`.

Q5 - When did you pivot from the browser approach?

Partial, recent pivot (~2-3 months), not complete. Unbrowse originally built a *commons for agents to browse the internet*. As agents/AI improved, they learned to browse anyway - diminishing returns on a route-commons. Inverting the idea - point the same route-engineering at *your computer* instead of the web - is Aiko, started for the Colosseum hackathon. Unbrowse still exists; the two are sister projects (R12). See `00-README`.

Q6 - If the EBM approach works, why wouldn't OpenAI/Anthropic copy it?

They would - it's copyable. The moat is **not the algorithm; it's the** ledger + distribution. **No one in AI has a distributed ledger the size of Solana** to hold all the weights, plus wallet-level distribution. Win the storage substrate and the network of corroborated weights before the technique is commoditized. **Honest:** this is a *race*, not a technical lock (R7). See `03-ONCHAIN`.

Q7 - How close are you to containing the energy usage?

Founder estimate: **~95% - the remaining work is stability, not invention.** In principle it can run fully on-chain and even remotely (not on your machine), because the design lets you push data on-chain by paying some SOL. **Honest** status: **the on-chain offload is** DESIGNED and partially proven **(real reads,** real vault, anchoring primitives) but the full "weights on-chain, laptop is a frecency cache" loop is not yet running at scale (R5). See `03`.

Q8 - How do you measure whether it's working?

The falsifiable atom: **does energy-ranked resolution beat scan-order at choosing** what fires next? **-** yes, measured: 2 passes vs 3, mutation-verified (`04-PROOF`). At the capability level, **ARC-AGI-3**: an earlier version reached a high score **without any large model** - the harness carried the reasoning. **Founder-reported, awaiting independent replication** (request the score, split, and a reproducible invocation).

Q9 - The one-shot game build - did you pre-train it? How did it learn?

No task-specific pre-training. At that time it was **harness + Opus** (the model as reasoner *inside* the harness), which **rebuilt a Wine layer to run** Overwatch on a Mac in one shot. **It works by forcing the model to think in** first principles **- decompose the route/travel problem layer by layer, locate where the** failure actually is, and fix it - made mechanical and instinctive by default, rather than free-associating. See `01`, `04`.

Q10 - Is this more a harness than a model? Is the harness the EBM?

Yes - it's primarily a harness, and the harness *is* the energy-based model. The EBM is the **recommendation/routing system** that learns which action to fire from collective history (BUILT+PROVEN). *Separately*, a **small non-parametric** reasoning model **is in training - kept as small as possible so it's local, and** made powerful by being **grounded** by the harness (retrieval + energy routing) rather than by parameter count. A small grounded model beats a same-size ungrounded one. See `02-ARCHITECTURE S6`.

Q11 - What's the next step if it works?

Beyond automating enormous amounts of work, the founder's stated end-vision is to **accelerate the creative process** and eventually **define a new currency** - one that rewards *value created / novelty*, not value-of-work. **Honest:** this is an early, unformed aspiration with real regulatory surface (R10) and should be de-scoped from the near-term investment thesis; the near-term is: contain the energy, ship to first users, prove the collective with more than one agent.

Q12 - What can we diligence right now?

- **This dossier** (00-06) - product, architecture grounded in real code, economics, on-chain/privacy, proof, and this FAQ.
- **The code** - the `arc-lean` tree; run `bash scripts/build.sh` (unit tests + 53 parity claims + signed attestation) to verify the **PROVEN** claims yourself.
- **The best next artifacts to request:** a recorded (Loom) demo of the harness one-shotting a fresh computer-use task, and the ARC-AGI-3 run with its exact numbers and a reproducible command.

Bottom line for the panel: the hard, cheap-to-test atom is proven (the router routes better than chance); the grand claim (reasoning-grade collective intelligence) is a named, ordered bet; the moat is a race to own a commons; and

every seam is labelled in `05-RISK-honest-wall.md`. This is a research-risk investment with an unusually disciplined honesty culture and a working, demonstrated harness - underwrite the founder and the falsifiable next gate, not the destination.

PLAN
On-Ch

Goal (user's words): store contracts on-chain where they *also act as EBM weights*; share data in the ledger to be space/cost efficient; store **primarily** on-chain**, local machine as a** frecency cache**. Plus: make the EBM actually rank which contract fires next** (energy-ranked resolution).

Corrected premise (verified this session - the earlier draft was wrong). The substrate already trains a real fractal EBM *on-fire* - but on `main`, not arc-lean, and via `ebm.zig`/`ebm_store.zig`/`jepa.zig` (NOT `energy_route.zig`, which is mislabeled dead code = `binary_skill`). What's actually there:

- **`ebm.zig`**: `E(x) = -cosine(x, centroid)`; the "weight" is a 256-d centroid `mean` + reward-mass `w`. Trained by a reward-weighted Hebbian fold on every declare/satisfy/die, with a 30-day freshness decay. Honest and shallow.
- **`jepa.zig`**: `E(x,y)=||W-x-y||2`, `W` a 256x256 matrix, real online SGD.
- **Fractal-atomic**: the *same* centroid atom is folded across a self-similar key fan per event - `invoke:{event}`, `:L{layer}`, `:dim:{w6h}`, `:verb:`, `:ptype:`, `:root`. Fractal at the indexing level; one flat atom per key.
- **Ledger-reconstructible**: `replayFromLedger` rebuilds all centroids purely from terminal verdicts (`satisfied`->`verdict:ok`, `died`->`verdict:retryable`). So the resolved ledger *is* the weights. Weights are ~1 KB per centroid key.

Two things are missing, and they are this plan:

1. **The energy never drives the fire decision.** `route`/`rankByEnergy`/`plan` (argmin-energy selection) exist and are unit-tested, but the live resolver only uses energy to *annotate* the next lever ("resolved-like") and to *gate* tool outputs (typicality). The wire from energy-ranking -> "what fires next" is absent.
2. **The weights live in the local ledger, not on-chain**, and are not cost-shared.

Why energy-ranking (question the requirement - the value, and its limits)

Value (where firing is expensive - i.e. on-chain / LLM):

1. **Best-first resolution** - fire the lowest-energy (most resolved-like) frontier contract first -> settle in fewer cycles, fewer wasted fires.
2. **Cost-aware spend** - energy says *where the SOL/LLM budget pays off* (`payment_energy`/`contract_budget` already score cheapest-chain-that-settles). This is the direct link to the cost goal.
3. **Waste/anomaly pruning** - skip candidates whose energy says "nothing that satisfied ever looked like this"; don't spend on dead paths.
4. **Self-improving, zero external training** - trains on-fire from the ledger.

Honest limits (state them or it becomes an over-claim):

- Shallow model (cosine-centroid + linear W) = a **soft prior, not a planner**.
- Must never **starve** a contract: energy *prioritizes*, never *forbids*; keep an exploration floor; HOLD stays non-terminal.
- Value is concentrated where firing is **expensive** - negligible for cheap local resolution. So: **prove the router helps (measured) before paying to** store weights on-chain.

Where this lives (a Phase-0 decision)

AVM Confidential - Prepared for investor due diligence - Not for distribution

`main` has the EBM organ (wired, ledger-reconstructible). arc-lean deleted it in the lean. Decide: build on `main` (organ present) vs. yoink the 3 EBM files onto arc-lean first. Do NOT assume - the branches diverged ~17k lines; pick with eyes open.

Risk register (flag before building)

1. **Shallow model** - set expectations: better-than-random routing, not reasoning.
2. **On-chain cost may be prohibitive** even deduped - Phase 0 measures; fork to commitment-on-chain + weights-off-chain (Arweave) if the numbers say so.
3. **Delta-compressibility unproven** - base+delta only saves money if contracts share a base. Verify on real weights before betting on it.
4. **Starvation/feedback loops** - a learned router can self-reinforce a bad prior. The fairness floor + exploration is not optional; it's a correctness invariant, mutation-tested.

Phases - prove value BEFORE paying for storage

Phase 0 - Measure, define, decide (no storage code)

- Confirm the energy definition (it exists - `ebm.zig`/`jepa.zig`); decide which fire decision it should drive (frontier order? candidate/verb selection? both?).
- **Baseline the resolver**: instrument current settle-time, fires-to-settle, and (where applicable) SOL/LLM spend on a fixed task set. This is what Phase 1 must beat.
- **Measure Solana rent per KB** (devnet + mainnet); size a real centroid (1 KB) and a real delta; build the cost model; decide on-chain/off-chain boundary.
- **Decide where** (main vs yoink to arc-lean).
- **Gate:** written energy->decision spec, a measured resolver baseline, a cost model with real numbers, a branch decision.

Phase 1 - Energy-ranking router (the missing wire) - LOCAL, prove value

- Wire `ebm_store.route` / `jepa` energy into the resolver's **choice of what** fires next**: score the unresolved frontier (and, where a contract has** alternative actions/verbs, the candidates) by energy; fire best-first.
- **Fairness floor (correctness invariant):** energy only *reorders*; every live contract still fires within a bounded number of cycles (no starvation); an exploration fraction fires off-prior to keep learning; HOLD stays non-terminal.
- **Gate (this is where "does it help" is PROVEN, not assumed):** on the Phase-0 task set, energy-ranked resolution beats scan-order on >=1 real metric (fires-to-settle | / settle-time | / wasted-spend |) by a stated margin, with the fairness floor mutation-tested (disable it -> a contract starves -> RED). If it does NOT beat baseline, STOP - the router isn't worth wiring, and no on-chain storage follows.

Phase 2 - Content-addressed on-chain weight store + free dedup

(only if Phase 1 proved value AND Phase 0 cost is feasible)

- Content-addressed chunks: `blob=(sha256(bytes),bytes)`; **PDA = derive(hash)** -> identical content = same account = write-if-absent = **dedup for free** (the "share data" cost win falls out of content-addressing).
- Store/read a chunk on-chain (extend `iq_ledger`, or IQLabs `codeIn`-style chunking / adopt that brick). Local CAS cache keyed by hash.
- **Gate:** store a blob, store it again = 0 new on-chain bytes; evict local, read back from chain = identical; cost-per-unique-KB measured; corrupt a chunk -> hash-mismatch caught.

Phase 3 - Base + delta EBM (cost lever)

Aiko - Confidential - Prepared for investor diligence. Not for distribution.

- Shared base centroids stored once; per-contract weights = base_ref + small delta. Contract row carries `{lineage, pointers, base_ref, delta_ref, commit}`.
- **Gate:** reconstructed weights == full; two contracts sharing a base pay only their deltas (measured); inference/routing output correct; wrong delta -> RED. Also *verifies* the delta-compressibility assumption (risk #3).

Phase 4 - Frecency cache (chain canonical, local hot)

- Local cache with frecency eviction (frequency x recency decay), sidecar index; miss -> fetch chain; hit -> local. Tiers: hot=full local, warm=commit+fetch, cold=commit-only.
- **Gate:** `rm -rf cache && refetch -> identical`; hot survives, cold evicts, size bounded; measured hit-rate on a real access trace; cold miss + no network -> HOLD (never fabricate).

Phase 5 - Compression + quantization (measured byte savings)

- gzip/zstd + int8/int4 shards, stacked with dedup+delta.
- **Gate:** measured on-chain-bytes reduction vs Phase-2 baseline; correctness within a stated tolerance; routing output unchanged beyond tolerance.

Phase 6 - Confidentiality (optional; wires the earlier discussion)

- Encrypt chunks (`payload_crypto` envelope / IQLabs `multiEncrypt`) -> on-chain ciphertext; receptor grant gates key release; PRE for dynamic grants; ZK only for property/anonymity - not for storage.
- **Gate:** unauthorized can't decrypt; authorized can; dedup/cost preserved.

Cross-cutting invariants

- **Prove value before paying for storage** (Phase 1 gate is the fork).
- **Fairness floor:** energy prioritizes, never starves; exploration keeps learning; HOLD non-terminal. Mutation-tested.
- **Chain canonical, local reconstructable:** every phase survives `rm -rf cache && refetch -> identical`.
- **Agency spend-gate first** (`agency.zig`): every on-chain write checks the SOL budget before signing, or a routing/store loop drains the wallet.
- **Everything mutation-proven + measured.** No un-measured estimate ships; no test that can't fail. Honest naming (say "centroid"/"linear"/"encrypted", not "zk").

Phase 0 RESULTS (measured 2026-07-15, mainnet RPC + deterministic formula agree)

Rent-exempt minimum (a recoverable DEPOSIT, not a burn; per-write fee ~0.000005 SOL):

data size	SOL locked	~USD @ $150
32 B (a commitment hash)	0.00100	$0.15
1 KB (a centroid)	0.00802	$1.20
4 KB	0.0294	$4.41
8 KB	0.0579	$8.69
256 KB (JEPA matrix)	~1.83	~$274

Boundary decision (grounded in these numbers):

- **On-chain (store the bytes):** small contract rows + shared centroids (~1 KB) + per-contract weight *deltas* - anything **<= ~4 KB** (<= ~0.03 SOL locked). Free dedup via content-addressed PDA means a shared base/centroid is paid **once**.
- **Off-chain (commitment only):** any tensor **> ~4 KB** (the JEPA 256 KB matrix, real NN weights MB-GB) goes to Arweave/IPFS/local; only its **32-byte commitment** (~0.001 SOL) is anchored on-chain. On-chain storage of large tensors is infeasible

and unnecessary.

- **Frecency bounds locked capital to the working set:** hot contracts fully on-chain; cold = commitment-only, rehydrated on use. Locked SOL scales with the *hot set*, not total history.

Economics honest take: the burn is negligible; the number that matters is *locked capital*. For 10 K contracts at ~300 B on-chain each (base+delta+dedup): ~30 SOL locked (~$4.5 K, recoverable) + ~$8 in fees. Feasible and dedup/delta/frecency are exactly the levers that shrink it. "Contracts as EBM weights on-chain" is viable **because the weights are ~1 KB centroids, not big tensors** - the big tensors stay off-chain by commitment.

Where: build on arc-lean; yoink main's self-contained 9-file Solana engine (analysis: zero calls into main's core; rests on `id`/`keystore` arc-lean already has). Port the `agency.zig` spend-gate FIRST.

Build spine

`0 measure/define` -> `1 energy router (prove value)` -> *fork: worth it?* -> `2 CAS+dedup` -> `3 base+delta` -> `4 frecency cache` -> `5 compress/quant` -> `6 confidentiality`. Phase 1 is the load-bearing proof; 2-6 only follow if it pays.